PROJECT REPORT

DESIGN AND DEVELOPMENT OF A BOOK MANAGEMENT SYSTEM IN C

A menu-based program using structures and arrays

Submitted by	Pari Goyal
Name	Pari Goyal
UID / Roll Number	26BCD10314
Course / Programme	BCA
Subject	C Programming
Submitted to	Kamaldeep Kaur
Academic Session	2026

BACHELOR OF COMPUTER APPLICATIONS

01 / OVERVIEW

Project Overview

Purpose, problem statement, and learning objectives

ABSTRACT

The Book Management System is a small C program for keeping a list of books during one program session. For each book, it stores a title, an author, and an availability status. A menu lets the user add a book, see the full list, find a book by its exact title, or change its status.

Problem Statement

A short book list kept on paper can be inconvenient to search and update. This project demonstrates how a simple computer program can keep related book details together and offer a few basic management actions through a clear menu.

Project Objectives

Learn core C ideas

Use a structure and an array to store multiple records with related fields.

Practise program flow

Use a loop, a switch statement, and if-else conditions to respond to menu choices.

Use functions

Keep adding, viewing, searching, and updating work in separate functions.

Handle text details

Read titles and author names that can contain spaces, and compare titles with strcmp.

Expected Outcome

When the program runs, the user can create and manage a small in-memory book list. It is intended as a beginner-level console project, so the actions and data structure are visible in the source code.

02 / SCOPE

Features and Requirements

What the program supports and where its limits are

Option	What it does
Add a book	Reads a title and author, then marks the new book as available.
View all books	Prints each saved title, author, and current availability status.
Search a book	Looks for a book by an exact title match and displays its details.
Update availability	Changes the matching book's status to Available or Not available.
Exit	Ends the program.

Requirements

Software

C compiler such as GCC; a terminal; and a text editor such as Visual Studio Code.

Hardware

A standard computer capable of running a C compiler. No special hardware is required.

Project Limits

- The program can hold up to 50 books at one time.
- Each title and author field can hold up to 49 visible characters.
- The book list is kept in memory and is cleared when the program closes.
- Search uses an exact, case-sensitive title match; duplicate titles are not blocked.

SCOPE CHOICE

This version focuses on structures, arrays, functions, and control flow. File saving, deleting books, and issue or return records can be added as future improvements.

03 / DESIGN

Data and Program Flow

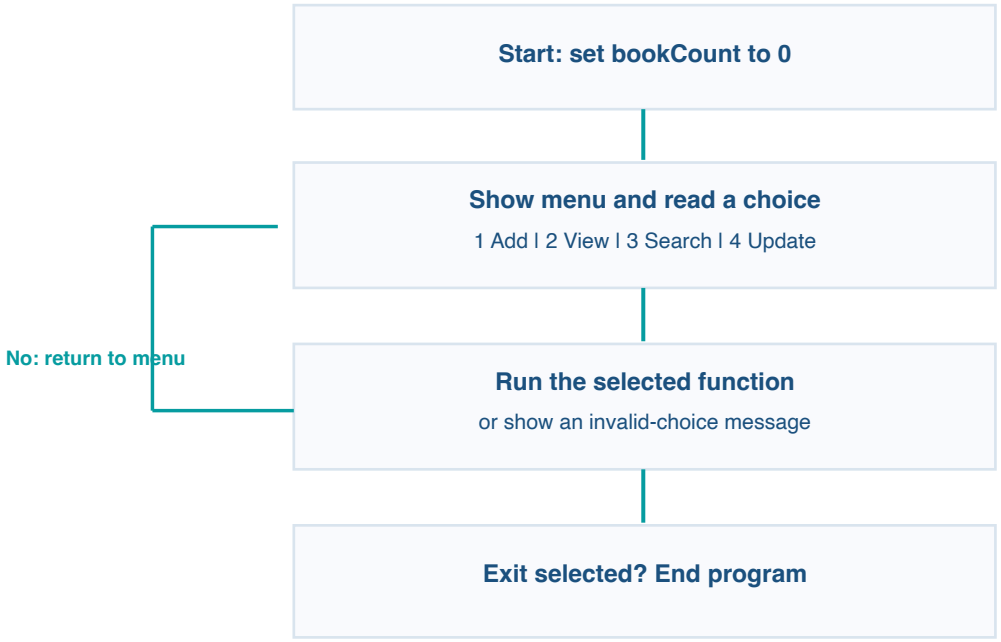
How book details are represented and how the menu repeats

Book Record

Field	C type	Meaning
title	char[50]	Book title, including spaces.
author	char[50]	Author name, including spaces.
available	int	1 means available; 0 means not available.

The program creates an array of 50 struct Book records. A separate integer, bookCount, tells the functions how many entries have been added so far.

Menu Flow



Each selected function returns to main. The while loop then displays the menu again. Choosing Exit ends the program.

04 / ALGORITHMS

How Each Option Works

Simple step-by-step logic used in the program

Add a book

1. Check that the 50-book limit is not reached.
2. Read the title and author.
3. Set availability to 1.
4. Increase bookCount by 1.

View all books

1. If bookCount is 0, show an empty-list message.
2. Loop through the saved entries.
3. Print each title, author, and status.

Search for a book

1. Read the exact title to find.
2. Compare it with each saved title using strcmp.
3. Show details when a match is found.
4. Show a not-found message if there is no match.

Update availability

1. Find the book by exact title.
2. Ask for status 1 or 2.
3. Store 1 for Available or 0 for Not available.
4. Show a confirmation message.

Functions in the Source Code

Function	Purpose
main	Displays the menu and calls the selected function.
addBook	Reads details and adds a record to the array.
viewBooks	Displays every record currently in the array.
searchBook	Finds and prints one record by its title.
updateAvailability	Changes the status for a matching title.

05 / USE

Compile, Run, and Review

Commands and an example interaction

Compile and Run

Open a terminal in the folder containing book_management.c, then compile the program:

```
gcc book_management.c -o book_management

macOS / Linux: ./book_management
Windows:       book_management.exe
```

Example Interaction

```
Enter your choice: 1
Enter book title: The Alchemist
Enter author name: Paulo Coelho
Book added successfully.

Enter your choice: 2
--- Book List ---
1. Title: The Alchemist
   Author: Paulo Coelho
   Status: Available

Enter your choice: 4
Enter the exact book title: The Alchemist
Choose the new status: 2
Book status updated.
```

Conclusion

This project demonstrates a small book list program using beginner C concepts. Its separate functions make each menu action easier to follow, and the structure keeps each book's details together.

Future Improvements

- Save and load the list from a file.
- Add delete, issue, and return options.
- Allow a search that ignores letter case.

APPENDIX / SOURCE CODE

Program Source Code (1 of 5)

Complete C source listing, continued in order from the previous page.

```
#include <stdio.h>
#include <string.h>

#define MAX_BOOKS 50
#define TEXT_SIZE 50

/* Each book has a title, an author, and an availability value. */
struct Book
{
    char title[TEXT_SIZE];
    char author[TEXT_SIZE];
    int available; /* 1 means available, 0 means not available */
};

/* Function declarations */
void addBook(struct Book books[], int *bookCount);
void viewBooks(const struct Book books[], int bookCount);
void searchBook(const struct Book books[], int bookCount);
void updateAvailability(struct Book books[], int bookCount);

int main(void)
{
    struct Book books[MAX_BOOKS];
    int bookCount = 0;
    int choice;

    /* Show the menu again after each action. */
    while (1)
    {
        printf("\n===== Book Management System =====\n");
        printf("1. Add a book\n");
        printf("2. View all books\n");
        printf("3. Search for a book\n");
        printf("4. Update availability\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");

        /* Check that the menu choice is a number. */
        if (scanf("%d", &choice) != 1)
        {
            printf("Please enter a number from 1 to 5.\n");
            return 1;
        }

        switch (choice)
        {
            case 1:
                addBook(books, &bookCount);
```

APPENDIX / SOURCE CODE

Program Source Code (2 of 5)

Complete C source listing, continued in order from the previous page.

```
                break;

            case 2:
                viewBooks(books, bookCount);
                break;

            case 3:
                searchBook(books, bookCount);
                break;

            case 4:
                updateAvailability(books, bookCount);
                break;

            case 5:
                printf("Program closed.\n");
                return 0;

            default:
                printf("Please choose an option from 1 to 5.\n");
        }
    }
}

/* Read a book's details and add it to the array. */
void addBook(struct Book books[], int *bookCount)
{
    if (*bookCount == MAX_BOOKS)
    {
        printf("The book list is full.\n");
        return;
    }

    printf("Enter book title: ");
    /* The space before %49 skips the Enter key left by the menu input. */
    if (scanf(" %49[^\n]", books[*bookCount].title) != 1)
    {
        printf("Could not read the title. Please try again.\n");
        return;
    }

    printf("Enter author name: ");
    if (scanf(" %49[^\n]", books[*bookCount].author) != 1)
    {
        printf("Could not read the author name. Please try again.\n");
        return;
    }
```

APPENDIX / SOURCE CODE

Program Source Code (3 of 5)

Complete C source listing, continued in order from the previous page.

```
    /* New books are available when they are first added. */
    books[*bookCount].available = 1;
    *bookCount = *bookCount + 1;

    printf("Book added successfully.\n");
}

/* Print every book that has been added. */
void viewBooks(const struct Book books[], int bookCount)
{
    int i;

    if (bookCount == 0)
    {
        printf("No books have been added yet.\n");
        return;
    }

    printf("\n--- Book List ---\n");
    for (i = 0; i < bookCount; i++)
    {
        printf("%d. Title: %s\n", i + 1, books[i].title);
        printf("   Author: %s\n", books[i].author);

        if (books[i].available == 1)
        {
            printf("   Status: Available\n");
        }
        else
        {
            printf("   Status: Not available\n");
        }
    }
}

/* Find and display a book by its exact title. */
void searchBook(const struct Book books[], int bookCount)
{
    char titleToFind[TEXT_SIZE];
    int i;

    if (bookCount == 0)
    {
        printf("No books are available to search.\n");
        return;
    }

    printf("Enter the exact book title: ");
```

APPENDIX / SOURCE CODE

Program Source Code (4 of 5)

Complete C source listing, continued in order from the previous page.

```
    if (scanf(" %49[^\n]", titleToFind) != 1)
    {
        printf("Could not read the title. Please try again.\n");
        return;
    }

    for (i = 0; i < bookCount; i++)
    {
        if (strcmp(books[i].title, titleToFind) == 0)
        {
            printf("Title: %s\n", books[i].title);
            printf("Author: %s\n", books[i].author);

            if (books[i].available == 1)
            {
                printf("Status: Available\n");
            }
            else
            {
                printf("Status: Not available\n");
            }
            return;
        }
    }

    printf("Book not found. Check the title and try again.\n");
}

/* Change a book's status to available or not available. */
void updateAvailability(struct Book books[], int bookCount)
{
    char titleToFind[TEXT_SIZE];
    int newStatus;
    int i;

    if (bookCount == 0)
    {
        printf("No books are available to update.\n");
        return;
    }

    printf("Enter the exact book title: ");
    if (scanf(" %49[^\n]", titleToFind) != 1)
    {
        printf("Could not read the title. Please try again.\n");
        return;
    }
```

APPENDIX / SOURCE CODE

Program Source Code (5 of 5)

Complete C source listing, continued in order from the previous page.

```
    for (i = 0; i < bookCount; i++)
    {
        if (strcmp(books[i].title, titleToFind) == 0)
        {
            printf("1. Available\n");
            printf("2. Not available\n");
            printf("Choose the new status: ");

            if (scanf("%d", &newStatus) != 1 ||
                (newStatus != 1 && newStatus != 2))
            {
                printf("Please enter 1 or 2.\n");
                return;
            }

            if (newStatus == 1)
            {
                books[i].available = 1;
            }
            else
            {
                books[i].available = 0;
            }

            printf("Book status updated.\n");
            return;
        }
    }

    printf("Book not found. Check the title and try again.\n");
}
```